FIELD GUIDE FG-002

WAFT FIELD GUIDE

Level 2: Professional Developer's Guide

TECHNICAL

Issued by: WAFT Engineering Team
Date: January 11, 2026

INTRODUCTION

This guide provides technical details for developers and engineers working with WAFT. It covers architecture, APIs, integration patterns, and best practices for building production systems with WAFT.

> ⚠ **PREREQUISITES**
>
> This guide assumes familiarity with Python, command-line tools, and software architecture. If you're new to WAFT, start with Level 1: Layman's Guide.

ARCHITECTURE OVERVIEW

Three-Layer Architecture

WAFT uses a three-layer architecture:

1. **Substrate Layer:** Package management (uv), project structure, dependency management. This is the foundation.
2. **Memory Layer:** Knowledge organization (_pyrite directory), active work, backlog, standards. This stores project state.
3. **Agents Layer:** AI agent capabilities (optional CrewAI integration), evolutionary systems, fitness testing. This is where agents operate.

Core Components

Table 1: WAFT Core Components

Component	Location	Purpose
Substrate Manager	`core/substrate.py`	Manages uv operations, pyproject.toml
Memory System	`_pyrite/`	Organizes active/backlog/standards
Agent Framework	`core/agent/`	BaseAgent, AgentState, AgentConfig
World System	`core/world/`	Biome, PetriDish environments
Science Module	`core/science/`	TheObserver, Taxonomy, Reports
Gamification	`core/gamification/`	RPG-style progression system

API REFERENCE

Command-Line Interface

CORE COMMANDS

- ☐ `waft new <name>` - Create new project
- ☐ `waft verify` - Verify project structure
- ☐ `waft sync` - Sync dependencies
- ☐ `waft add <package>` - Add dependency
- ☐ `waft init` - Initialize in existing project
- ☐ `waft info` - Show project information

Python API

Creating an Agent

```
from waft.core.agent import BaseAgent, AgentConfig

config = AgentConfig(
    name="MyAgent",
    description="An agent that solves problems"
)
```

```
agent = BaseAgent(config=config)
agent.initialize()
```

Document Generation

```
from waft.templates.field_guide import generate_field_guide
from pathlib import Path

generate_field_guide(
    title="My Guide",
    content="<h2>Content</h2><p>HTML content here</p>",
    output_path=Path("output.pdf")
)
```

Binder System

```
from waft.binder import Binder, DocumentEntry
from pathlib import Path

binder = Binder(
    title="My Binder",
    subtitle="Document Collection"
)

section = binder.add_section("Section 1")
section.add_document(DocumentEntry(
    path=Path("doc1.pdf"),
    title="Document 1"
))

binder.generate(Path("binder.pdf"))
```

INTEGRATION PATTERNS

Pattern 1: New Project Setup

1. Run `waft new project_name`
2. Review generated `pyproject.toml`
3. Check `_pyrite/active/` for initial work items
4. Run `waft verify` to confirm structure

Pattern 2: Adding Document Generation

1. Import desired template (e.g., `from waft.templates.field_guide import generate_field_guide`)

2. Prepare HTML content
3. Call template function with content and output path
4. Verify generated PDF

Pattern 3: Self-Documentation Loop

1. Use `waft.reflection.ReflectionSystem` to analyze codebase
2. Generate documentation using WAFT templates
3. Review generated docs to inform development
4. Repeat as codebase evolves

WORKFLOW PROCEDURES

Development Workflow

STANDARD DEVELOPMENT PROCESS

- [ ] Create or update work items in `_pyrite/active/`
- [ ] Implement features following WAFT patterns
- [ ] Run `waft verify` before committing
- [ ] Generate documentation for new features
- [ ] Update `_pyrite/standards/` if patterns change

Documentation Workflow

1. Identify documentation gaps using reflection system
2. Select appropriate template (field guide, lab notes, etc.)
3. Generate documentation PDF
4. Review and iterate on content
5. Add to binder if part of larger collection

TROUBLESHOOTING

Table 2: Common Issues and Solutions

Issue	Cause	Solution
Template generation fails	Missing WeasyPrint dependencies	Run `uv sync` to install dependencies
Binder merge errors	Corrupted PDF files	Regenerate source PDFs
Memory system not found	Missing `_pyrite/` directory	Run `waft init` to create structure
Agent initialization fails	Invalid configuration	Check `AgentConfig` parameters

PERFORMANCE OPTIMIZATION

Document Generation

- Use HTML content blocks efficiently (avoid excessive nesting)
- Cache template rendering for repeated content
- Generate PDFs in batch when possible
- Use appropriate template for content type

Binder Assembly

- Generate individual PDFs first, then combine
- Use temporary directory for intermediate files
- Clean up temporary files after binder generation
- Consider page count limits for large binders

BEST PRACTICES

DEVELOPMENT BEST PRACTICES

- [ ] Always run `waft verify` before committing
- [ ] Keep `_pyrite/` structure organized
- [ ] Document new features using WAFT templates
- [ ] Follow existing code patterns and conventions
- [ ] Test document generation in CI/CD pipeline

PRO TIP

Use WAFT's self-documentation capabilities to keep documentation in sync with code changes. The reflection system can identify when documentation needs updates.

ADVANCED FEATURES

Template Customization

All templates use Jinja2 for rendering. You can customize templates by:

- Modifying template files in `src/waft/templates/`
- Passing custom parameters to template functions
- Creating new templates following existing patterns

Binder Customization

Binders support custom styling, section dividers, and front/back matter. See `src/waft/binder.py` for full API documentation.

NEXT STEPS

1. Review Level 3: ML AI Scientist Guide for research methodology
2. Explore WAFT source code in `src/waft/`
3. Check examples in `examples/` directory
4. Join WAFT community for support and collaboration